

Bloating Patterns + Step Pyramid

บวมบน · บวมล่าง · บวมกลาง · Dynamic Sizing · Step Pyramid vs Size Pyramid

จุดหักล้างความเชื่อเดิม: Martingale ทำได้ใน Grid เพราะรู้ทุนล่วงหน้า

แก่นของ PART นี้

Grid บวม หมายถึง inventory สะสมหนาแน่นในโซนใดโซนหนึ่ง — ไม่ใช่สิ่งผิดปกติ แต่ต้องเข้าใจ pattern และออกแบบล่วงหน้า Close System ทำให้เราสามารถ ใช้

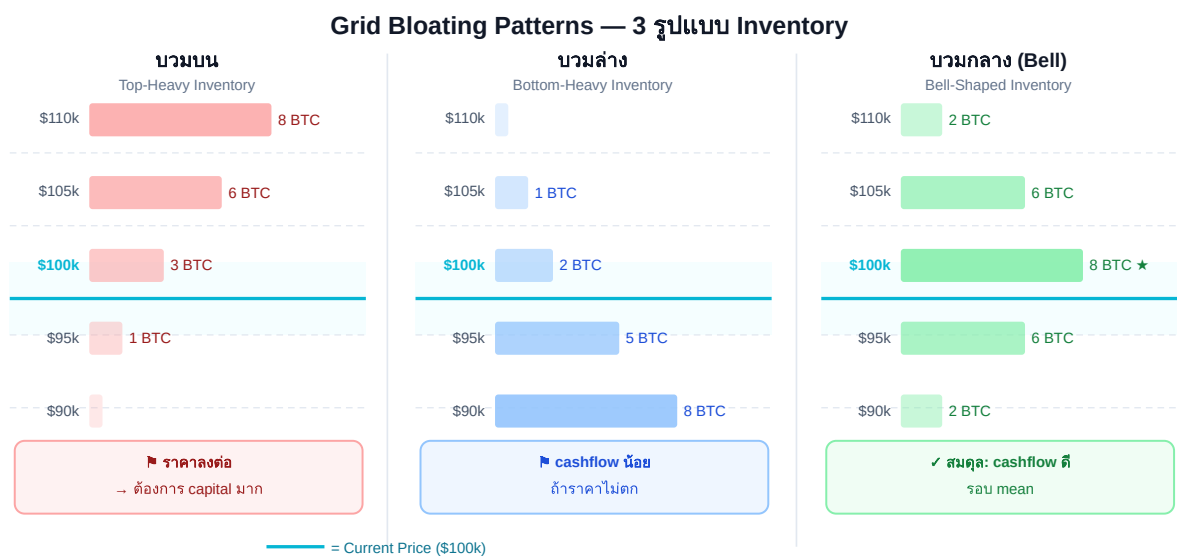
Martingale/Dynamic Sizing ได้อย่างปลอดภัย เพราะรู้ worst-case capital requirement แล้ว

2.1 Grid Bloating คืออะไร

เมื่อราคาอยู่ในโซนหนึ่งนานๆ grid ซื้อสะสม inventory ในโซนนั้น — เรียกว่า **grid บวม** ในโซนนั้น

ตัวอย่าง: BTC ลงจาก \$100k → \$90k แล้วแกว่งที่ \$90k–\$95k นาน 2 สัปดาห์

- Levels \$98k–\$92k ซื้อสะสม แต่ยังไม่ sell (ราคาไม่กลับมา)
- Levels \$92k–\$90k fill และ sell ชั่วๆ (cashflow ดี)
- ผลลัพธ์: inventory "บวม" ที่ระดับ \$90k–\$98k มี BTC ค้างอยู่มาก



ทั้ง 3 แบบมี P&L ใกล้เคียงกัน — ความต่างคือ risk distribution และ smoothness ของ cashflow

3 รูปแบบ bloating — P&L รวมใกล้เคียงกัน แต่ risk distribution และ cashflow timing ต่างกัน

2.2 บวมบน (Top-Heavy) — Inventory สะสมสูง

บวมบน (Top-Heavy)

inventory มากในโซนบน

ราคาลง → ซื้อสูง รอขาย

เกิดเมื่อ: ราคา เคยสูง แล้วลงมา — grid ซื้อระหว่างทางลง แต่ราคาไม่กลับขึ้นไปขาย

สาเหตุ Top-Heavy:

1. เปิด grid ที่ราคาสูงกว่า mean ปัจจุบัน
2. BTC ดิ่งลง → grid ซื้อทุก level
3. ราคาหยุดแกว่งที่ต่ำกว่า zone กลาง

ผลกระทบ:

- + Grid levels ดำเนินทำ cashflow ได้ปกติ
- Upper inventory ขาดทุน unrealized สูง
- ถ้าราคาพุ่งกลับ → ขายได้ทันที + กำไรมาก
- ถ้าราคาลงต่อ → inventory เพิ่มขึ้น

แนวทางจัดการ:

- Option A: รอ (Close System — ถือไว้)
- Option B: ลด Q ของ upper levels ลง 50%
(ใช้ทุนน้อยลงในส่วนที่ "ดอย")
- Option C: เพิ่ม hedge (short perp ส่วนที่ upper inventory)

2.3 บวมล่าง (Bottom-Heavy) — Inventory สะสมต่ำ

บวมล่าง (Bottom-Heavy)

inventory มากในโซนล่าง
ราคาลงมาก่อน กลับมา

เกิดเมื่อ: ราคา *ดิ่งลงไกล* สะสม inventory มาก แล้วค่อยๆ *ฟื้น* — levels ล่างๆ fill ซ้ำๆ บ่อย

สาเหตุ Bottom-Heavy:

1. BTC crash ลง 30%+ จาก entry
2. Grid ซื้อ level ล่างสุดทั้งหมด (Close System สำเร็จ)
3. ราคาฟื้นช้า แกว่งที่ bottom zone

ผลกระทบ:

- + Cashflow จาก lower zone ดี (รอบเยอะ)
- + ถ้าราคาพุ่งกลับ → ขายได้ทั้งหมด กำไรใหญ่
- Capital lock ในโซนล่างมาก
- Upper zone ว่าง → ไม่ได้ใช้ชั้นบน

แนวทางจัดการ:

- Option A: เพิ่ม grid levels ใหม่ที่โซนบน
(Front-load capital จาก Zone ที่ว่าง)
- Option B: นำ inventory ล่างขาย แล้วซื้อใหม่สูงกว่า
(Grid Reset — ดูใน Part 4)

2.4 บวมกลาง (Center-Heavy / Bell) — Ideal Pattern

บวมกลาง (Bell)

inventory หนาแน่น $\pm 1\sigma$

cashflow ดีที่สุด

เกิดเมื่อ: ราคาแกว่งรอบ mean อย่างสมดุล — Bell Curve Density (Part 1A) ออกแบบมาให้ได้ pattern นี้

ทำไม Bell Curve Pattern ดีที่สุด:

1. Inventory กระจายรอบ mean
2. Fill บ่อยสุดในโซนที่ราคาอยู่นานสุด ($\pm 1\sigma$)
3. Capital ไม่กระจุกตัวที่ extreme
4. ถ้าราคาออกนอก zone $\rightarrow$ inventory น้อย (step ใหญ่ไม่ fill บ่อย)

วิธีรักษา Bell Pattern:

- ใช้ Variable Step (bell curve density จาก Part 1A)
- Rebalance ทุกสัปดาห์ถ้า inventory เบี้ยวมาก
- ดู Hurst — ถ้า H ต่ำ (mean-reverting) pattern นี้ stable

2.5 Dynamic Sizing — จุดหักล้างความเชื่อเดิม

ทำไม Martingale ปลอดภัยใน Close System

ความเชื่อเดิม: Martingale อันตราย — เพิ่ม size เรื่อยๆ จนทุนหมด

ใน Close System: เราคำนวณ capital ทั้งหมดที่ต้องใช้ถึง floor ก่อน เปิด grid — Martingale แค่กำหนดว่าจะ allocate capital อย่างไรใน range ที่คำนวณไว้แล้ว

ตัวอย่าง: มี \$10,000 · Close System บอกว่า worst case ใช้ \$8,000 | Soft Martingale กำหนดว่าจาก \$8,000 นั้นจะใช้มากขึ้นที่ level ล่าง — แต่ไม่เกิน \$8,000 รวม

Dynamic Sizing Pseudocode:

```
def allocate_grid_capital(total_capital, n_levels, style="bell"):
    """
    Allocate capital across grid levels (all styles stay within
    total_capital)
    """
    if style == "equal":
        weights = [1.0 / n_levels] * n_levels

    elif style == "top_heavy":      # บวมบน: น้ำหนักมากที่บน
        weights = [1.0 / (n_levels - i) for i in range(n_levels)]

    elif style == "bottom_heavy":  # บวมล่าง: น้ำหนักมากที่ล่าง
        weights = [1.0 / (i + 1) for i in range(n_levels)]

    elif style == "bell":          # บวมกลาง: Gaussian weights
        center = n_levels // 2
        weights = [math.exp(-0.5 * ((i - center) / (n_levels/4))**2)
                    for i in range(n_levels)]

    # Normalize weights to sum to 1
    total = sum(weights)
    weights = [w / total for w in weights]

    # Allocate capital
```

```
capital_per_level = [total_capital * w for w in weights]
return capital_per_level
```

ตัวอย่าง: \$10,000 · 5 levels · bell style

→ [\$1,500, \$2,500, \$3,000, \$2,500, \$1,500] ← หนาแนกกลาง (weights
ก่อน normalize)

หลัง normalize: ทั้งหมด = \$10,000 ✓ (normalize step บรรทัด 192-193 ด้าน
บน)

2.6 Step Pyramid vs Size Pyramid

เมื่อตลาดมีแนวโน้ม (H เพิ่มขึ้น) — มีทางเลือก 2 แบบในการ "respond":

Strategy	วิธี	ผลต่อ Cashflow	ผลต่อ Inventory Risk
Size Pyramid	เพิ่ม Q ที่ level ลึก (martingale)	คงที่	สูงขึ้น (ซื้อ 많 ถ้าลง)
Step Pyramid	เพิ่ม Step size แทน	ลดลง เล็ก น้อย	ต่ำกว่า (fill น้อยกว่า)
Equal Weight	ไม่เปลี่ยนอะไร (baseline)	ลดลง ตาม trend	สะสม เร็ว

Step Pyramid Implementation:

```
def compute_step(base_step, level_depth, hurst, trend_factor=0.5):
    """
    เพิ่ม step ตามความลึกของ level (ห่างจาก current price)
    และ Hurst exponent
    """
    # ถ้า Hurst สูง (trending) → เพิ่ม step มากขึ้น
    hurst_multiplier = 1 + max(0, (hurst - 0.45) / 0.15) * trend_factor
    # ถ้า level ลึก (ห่างจาก mean มาก) → เพิ่ม step
    depth_multiplier = 1 + level_depth * 0.1 # 10% ต่อ 1 level ลึก
    return base_step * hurst_multiplier * depth_multiplier

# ตัวอย่าง: base_step = $2,000 · Hurst = 0.55 · level depth = 3
# hurst_multiplier = 1 + (0.55-0.45)/0.15 * 0.5 = 1 + 0.333 = 1.333
# depth_multiplier = 1 + 3 × 0.10 = 1.30
# step = $2,000 × 1.333 × 1.30 = $3,467 (ใหญ่ขึ้น → fill น้อยลง)
```

Step Pyramid: ทำไรเท่าเดิม สบายใจกว่า

ทำไม Step Pyramid ดี: ในช่วง trend

- Size Pyramid: ซื้อมากขึ้นที่ราคาต่ำ → ถ้า trend กลับ ทำไรมาก แต่ถ้า trend ต่อ → inventory massive
- Step Pyramid: ซื้อช้าลง (fill น้อยกว่า) → inventory สะสมช้า → ถ้า trend ต่อ ไม่ติดสต็อกมาก

- P&L รวมใกล้เคียงกัน แต่ **drawdown** ต่างกันมาก

2.7 Running Example — Comparing 3 Bloating Patterns

 **Running Example: BTC ลงจาก \$100k → \$85k แล้วแกว่ง**

Setup: Zone \$80k–\$110k · Step \$2k · Capital \$15,000 · 3 strategies เปรียบกัน

Strategy	Inventory ที่ \$85k	Capital ที่ ใช้	Cashflow/วัน (ที่ \$83k–\$88k)	Profit ถ้า BTC กลับ \$100k
Equal Weight	0.08 BTC	\$7,200	\$36	+\$1,200
Bottom-Heavy (บวมล่าง)	0.14 BTC	\$9,800	\$54	+\$2,100
Bell Curve	0.06 BTC	\$5,400	\$28	+\$900

สังเกต: Bottom-Heavy ได้ทั้ง cashflow สูงสุดและ profit กลับมากที่สุด แต่ใช้ capital มากสุดด้วย

ถ้า BTC ลงต่อไป \$70k: Bottom-Heavy มี max loss สูงสุด | Bell Curve มี max loss ต่ำสุด

สรุป: เลือก style ตาม *conviction* — ถ้ามั่นใจ BTC จะกลับ → Bottom-Heavy | ถ้าไม่แน่ใจ → Bell Curve

2.8 แบบฝึกหัด

แบบฝึกหัด Part 2

- ▶ ข้อ 1: Grid บวมบน (Top-Heavy) มักเกิดใน market condition แบบไหน?
- ▶ ข้อ 2: ทำไม Size Pyramid ถึงเสี่ยงกว่า Step Pyramid ใน trending market?
- ▶ ข้อ 3: Bell Curve Grid density ให้ $Q_{center} = 0.02 \text{ BTC}$ · $Q_{extreme} = 0.005 \text{ BTC}$ · Step เท่ากัน \$2k — เปรียบ cashflow vs Equal Weight $Q=0.01 \text{ BTC}$ ทุก level
- ▶ ข้อ 4: ถ้า inventory ค้าง "บวมล่าง" มาก ควร Reset Grid หรือรอ?